Prototyping 5

Michael Phelan, Concordia Lutheran High School | Dr. T. Jenkyn, Western University |

In this prototype phase I worked on compacting the device. This included soldering the breadboard layout on a small perfboard, adding a battery and making the data transmission wireless (so it doesn't need to be connected to my computer at all times), and finally adding a buzzer so the device beeps when you go over your prescribed limit.

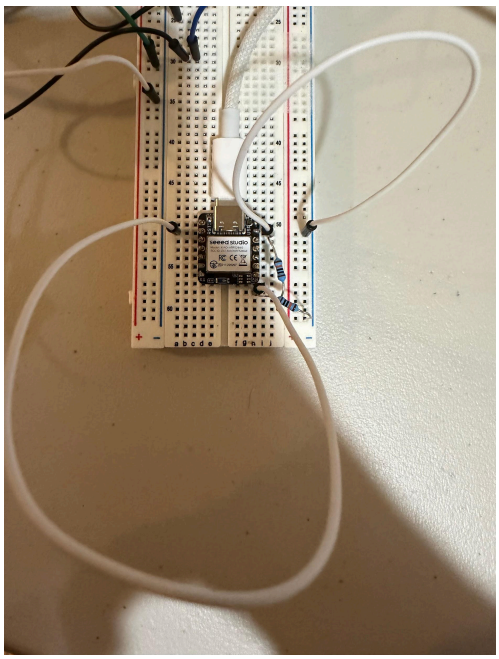
Moving from the Arduino UNO to the Seeed XIAO nRF52840

The Uno is way too big and bulky, so it was decided earlier to use the XIAO nRF52840. It was great because the XIAO can be uploaded on the Arduino IDE, so the application does not change.

However, the calibration curves are fitted against ADC values. They only transfer to a new board if that board produces the same raw numbers for the same physical force. This is not automatic as the XIAO defaults to a different resolution and a different voltage reference than the Uno. I was also planning on using a 3.7V LiPo, which is different from the 5V on the arduino. To make sure everything got transferred cleanly, these two lines of code were added:

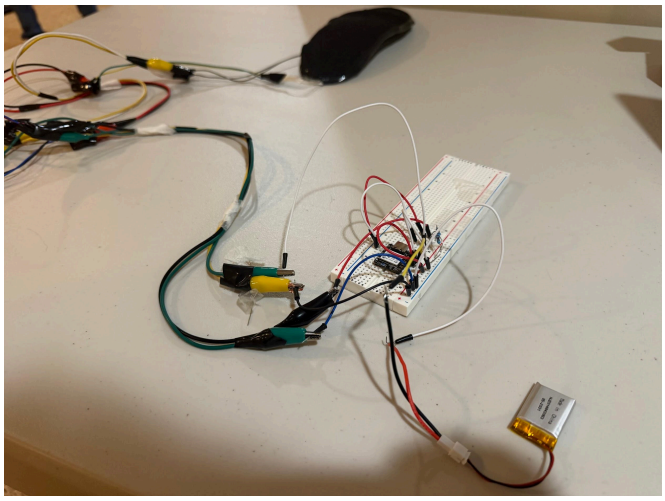
```
analogReadResolution(10); // 0..1023, matching the Uno

analogReference(AR_VDD4); // measure against supply, so the reading is a ratio
```



I verified this by making a simple circuit. A 100 kΩ / 100 kΩ divider read 509 on both boards, against an ideal of 511, meaning the code worked, and the calibration could successfully be transferred to this new microcontroller

Making it wireless



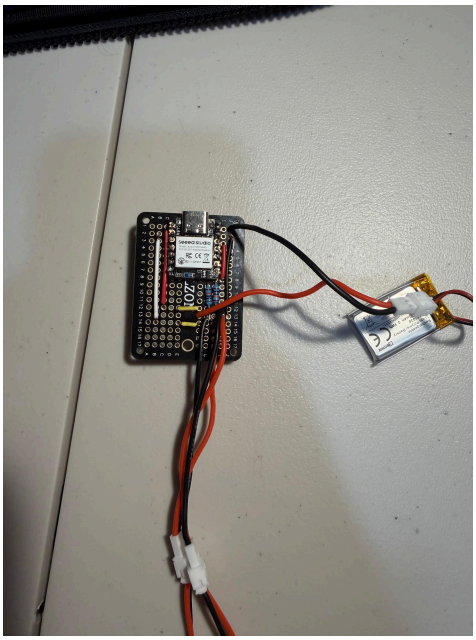
The next step was to re-wire the breadboard as it was before but instead use the XIAO instead of the Arduino UNO. I then attached a LiPo battery. It feeds the 5V / VBUS pin on the XIAO and passes through the board's own regulator. It works, which is great.

The XIAO has bluetooth, meaning it could connect to my phone. I first downloaded the nRF Connect app, but it would not show the stream of values, just a single one. So instead, I downloaded Adafruit Bluefruit Connect, which worked amazingly.

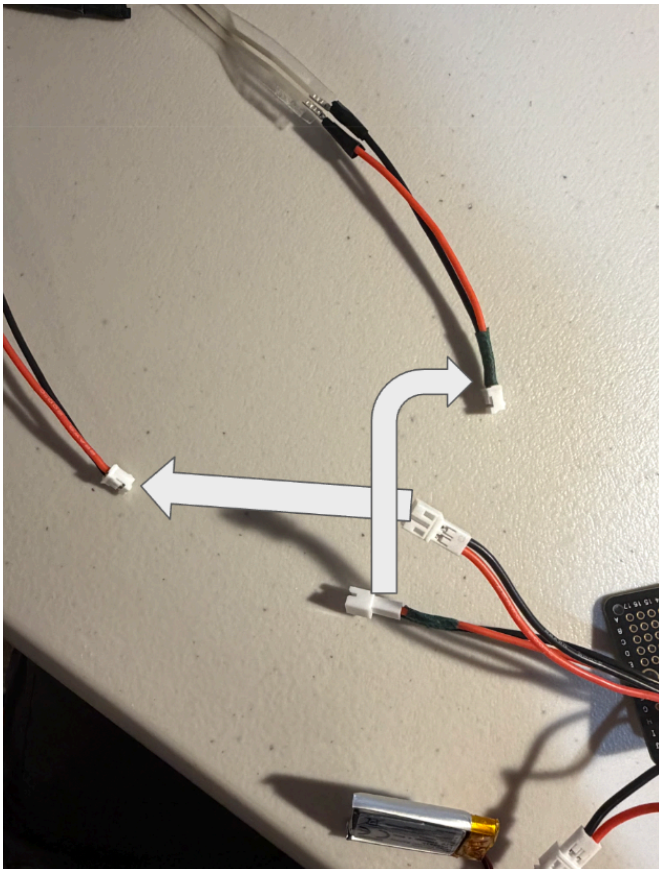
What was shown on my phone —->

```
15:19:29 RX ,
15:19:30 RX 31.3
15:19:30 RX ,
15:19:30 RX 9.5
15:19:30 RX
15:19:30 RX 92.5
15:19:30 RX ,
15:19:30 RX 26.9
15:19:30 RX ,
15:19:30 RX 9.7
15:19:30 RX
15:19:30 RX ,
15:19:30 RX 22.2
15:19:30 RX ,
```

Soldering onto Perfboard



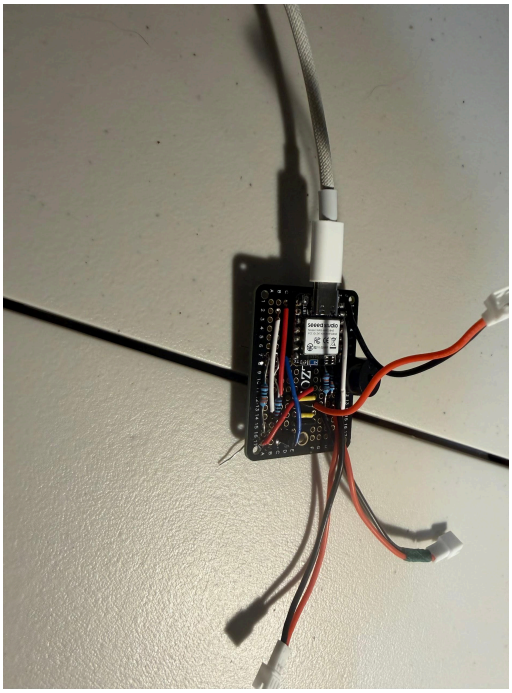
Now that I had the bluetooth/battery working correctly, I soldered everything onto a perfboard. The perfboard had conductive strips going across the letter rows, so I soldered matching the layout from the breadboard exactly. This was one of my first times soldering, so it was a little messy. However, I had a lot of fun and learned a lot.

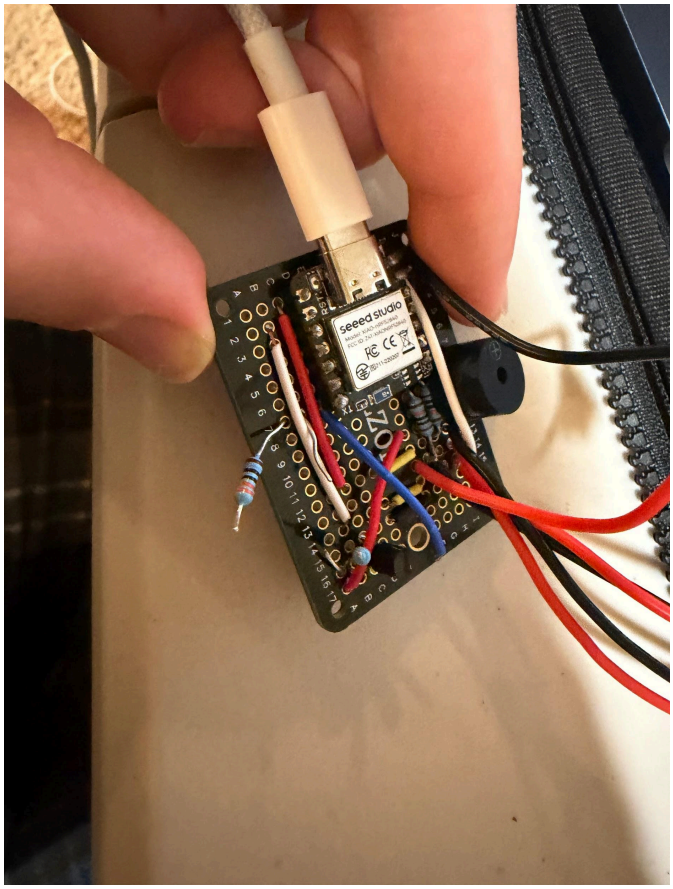


I used JST wires for the battery and to connect the ball and heel sensors to the perfboard so they are easily removable and can snap back on. I also soldered the other ends of the JST wires to the sensors. I put green tape on the end of the wires that corresponded to the heel wire, so I wouldn't get mixed up. See the picture on the left.

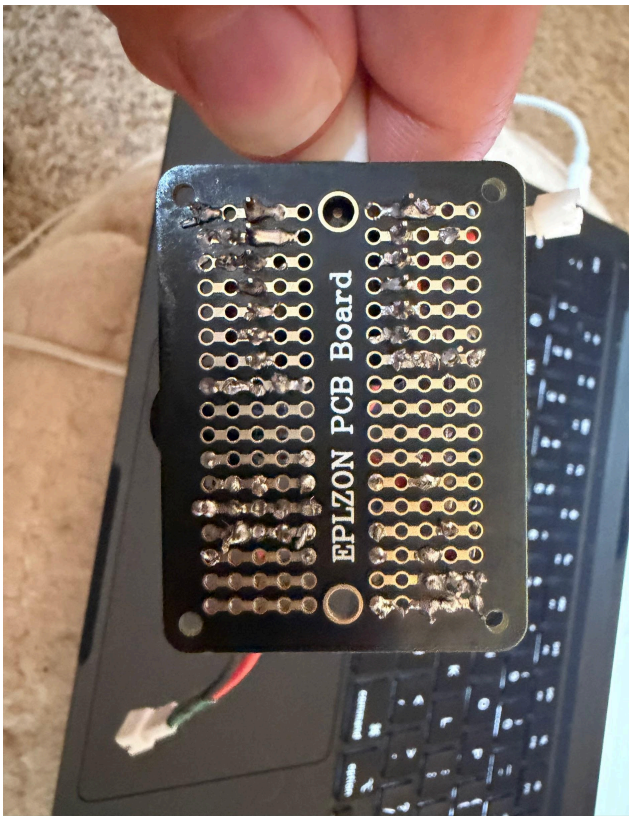
Adding an alarm

At this point, I realized that it would be a good idea to add a physical alarm to the perfboard, so when someone goes over the limit, it blares. I first tested an active buzzer on the breadboard, and it worked perfectly. Then I transferred to the perfboard. I put the buzzer in J8 (+) and J11 (-), a wire from F11 to A17, the PN2222 transistor in C15-C17, and the base resistor from A7 to A16. I tried adding another resistor in parallel but it didn't work. I even added another wire from C7 to E17, because I heard it beep. But that was a bad idea. It was connected to D6, which was the same pin that fed the transistor's base through the resistors. This meant that it beeped no matter what.





I eventually snipped the wire and resistors, because they were not working. Sometimes (when I was running the test alarm script), it would beep out of the blue, like when I was touching the board in a certain spot. But what really frustrated me was after I snipped the resistors, when the code was running I would touch them to each other, and it would beep. When I soldered them back on a second time, the beeping would stop and fail. Eventually, it even affected the sensor readings. The ball sensor's resting reading decreased from the typical 1010 to 910.



I then found the solution: the microcontroller's header pins were not soldered all the way down. Roughly half of them had a visible gap between the pin and the pad. This entire time I thought that it was perfectly fine, because some part of it was touching the metal (rookie mistake). But I realized that because of this, some signal was probably leaking through. I resoldered everything down completely, and cleaned all the flux residue off with isopropyl alcohol and a toothbrush. This instantly fixed the alarm outright. This also explained the part that had frustrated me most. Holding the resistors together by hand worked because holding them pressed the board down and closed the gaps. Both sensor channels also returned to a steady 1010 and 1012 unloaded, and the ball sensor stopped drifting.

Everything now soldered down fully
(I know it looks bad)

The code was changed to include the alarm system. Now, during startup, it asks you your prescribed partial weight bearing (in bodyweight %)

Insole

The insole was degraded from the lab testing, so I attached a fresh foam layer on top.



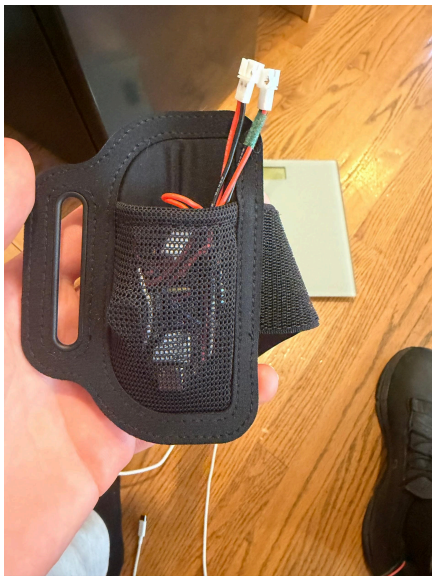


After around 2 hours of testing with this insole, it wore down a bit too. This is something that needs to be looked at in the future, as we cannot have an insole that wears down after a few uses.

Putting into an Anklet

Now that everything is compacted, I found a Nintendo Switch Sports ankle strap in my basement, and fit the electronics inside it.





Here is a picture of it all hooked up and worn in my shoe.



In summary, the device is now compacted into an anklet, can communicate wirelessly to my phone, and blares an alarm when you go above your prescribed limit.